

ANCHOR: Asset-Informed Geometric Skills for Language-Conditioned Manipulation

Anonymous Authors

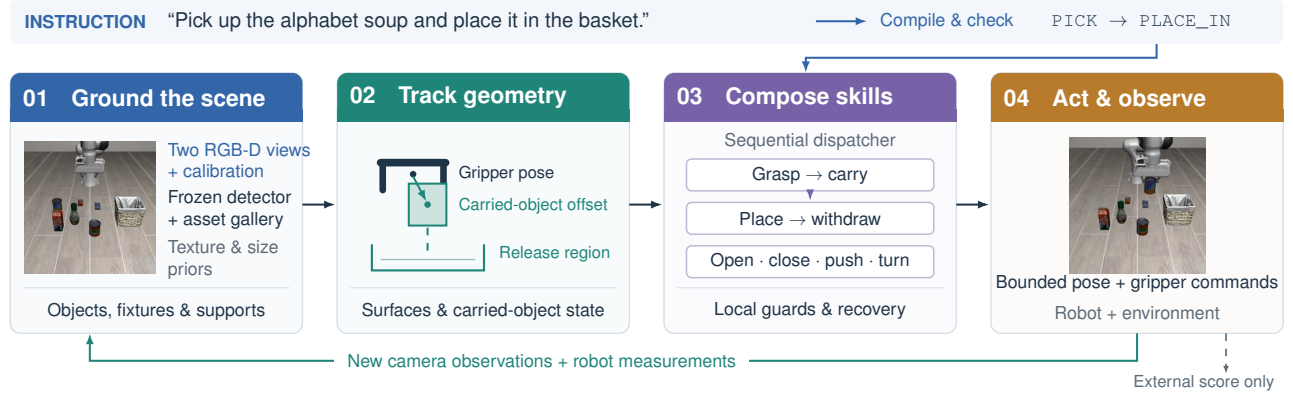


Fig. 1. ANCHOR grounds language in calibrated observations and asset priors, maintains carried-object geometry, and composes grasp-place and contact skills in a feedback loop. Insets are upright frames from an actual Object 00 rollout; the geometry drawing is schematic. Benchmark success is recorded externally and is not fed back to the policy.

Abstract—Language-conditioned manipulation requires semantic grounding, geometric reasoning, and reliable transitions between physical interactions. We study how far an explicit, asset-informed system can address these requirements without training an action policy on robot demonstrations. ANCHOR combines a constrained language compiler, frozen visual grounding, calibrated RGB-D reconstruction, and a sequential dispatcher for grasp-place and contact skills. Its geometric state distinguishes observed surfaces from offline object priors and maintains the displacement between the gripper and the carried object during placement. Contact skills use robot measurements and visual checks to manage articulation and recovery. The policy interface excludes task-success feedback, while an external evaluator records whether the official predicate is ever satisfied. On 400 fixed initializations spanning 40 LIBERO tasks, the deployed system succeeds in 360 trials (90.0%). Controller completion and external success disagree in 44 trials, demonstrating why these outcomes must be reported separately. The study characterizes a reproducible geometric baseline with explicit asset and calibration assumptions and places its measured outcomes alongside published action-policy results.

I. INTRODUCTION

Natural-language instructions provide a compact description of a manipulation goal, but leave much of the physical procedure unspecified. To place a bowl inside a drawer, a robot must identify the intended object, infer accessible geometry, establish a stable grasp, maintain clearance during transport, and release the object in a suitable region. Articulation introduces an additional difficulty: the robot must distinguish productive motion from mechanical resistance and decide when to release or reacquire contact. A plausible semantic interpretation alone does not resolve these questions.

Large vision-language-action policies offer one route to this capability. OpenVLA [1] adapts pretrained representations to action prediction, while SmolVLA [2] studies a more compact architecture and efficient execution. These approaches motivate an accompanying question: what performance can an explicit system achieve when visual grounding is pretrained but the action policy is constructed from geometry and hand-engineered skills? Answering this question requires disclosing the information available to the system, including asset knowledge and calibration, rather than equating the absence of policy training with the absence of prior information.

We present ANCHOR (Fig. 1), an asset-informed baseline for language-conditioned tabletop manipulation. The system compiles supported instructions into ordered skill programs and binds their entities to geometry reconstructed from two calibrated RGB-D views. It uses a frozen detector together with a gallery derived from public simulator assets. A dispatcher preserves state across grasp-place segments and invokes specialized contact controllers for drawers, doors, knobs, and pushing. Robot actions are bounded Cartesian pose increments with a gripper command.

The central engineering issue is continuity across physical stages. A semantically correct target can still be unreachable with the current wrist orientation; a successful close command need not establish retention; and a gripper placed at a receptacle center need not center the object being carried. ANCHOR therefore maintains explicit object-to-gripper displacement, re-observes relevant geometry at selected transitions, and uses local completion checks before advancing its program. These checks are fallible estimates, separate from benchmark success.

We evaluate the deployed implementation on the Spatial, Object, Goal, and Long suites of LIBERO [3]. Each suite contains ten evaluated tasks, with ten official initial states per task. The evaluation uses the existing controller and fixed evaluation configuration; its scope is a system study on known benchmark assets and instruction forms. We make three contributions:

- 1) An integrated language-to-geometry execution architecture that declares its offline priors and separates policy observations from external scoring.
- 2) A documented account of how carried-object geometry, visual checks, and contact-specific controllers are composed in the deployed system.
- 3) A record-based evaluation reporting task-level outcomes, execution costs, and disagreement between internal completion and external success.

The contribution is the integrated baseline and its empirical characterization, including the relationship between the controller’s local decisions and the benchmark’s externally observed outcomes.

II. RELATED WORK

A. Language, Programs, and Geometric Planning

Task and motion planning combines discrete decisions with continuous geometric feasibility [4]. Language-conditioned systems extend this interface: SayCan connects language choices to available skills and their affordances [5], while Code as Policies generates executable programs that call perception and control interfaces [6]. ANCHOR uses a deterministic grammar and a fixed skill vocabulary. This makes the represented program inspectable, but restricts language coverage to implemented constructions and validated entities.

VoxPoser constructs spatial value maps to guide motion [7]. ReKep expresses manipulation objectives through relational keypoint constraints [8]. These works show the utility of intermediate geometric representations. Our implementation uses object and fixture geometry, carried-object offsets, and specialized finite-state controllers. It does not synthesize arbitrary constraint programs or perform a unified optimization over the complete task. Its behavior depends on the coverage and calibration of its manually implemented skills.

B. Pretrained Perception and Action Policies

Grounding DINO supports text-conditioned object detection [9]. ANCHOR uses a frozen local checkpoint as one source of region proposals; the downstream geometric and gallery components remain essential to the implemented pipeline. In particular, public object textures and collision dimensions provide priors that are stronger than an image-only input setting.

OpenVLA [1] and SmolVLA [2] learn action generation from robot data. OpenVLA-OFT [10] studies action chunking, parallel decoding, continuous action prediction, and additional sensor inputs. ANCHOR provides a complementary, explicitly engineered control baseline. The distinction

concerns how actions are obtained; it does not imply an absence of learning in the pretrained detector or an absence of development on benchmark scenarios. Section V-A compares reported success rates while identifying differences in sensing, asset access, and evaluation conditions.

C. Benchmark Interpretation

LIBERO was introduced to study knowledge transfer in lifelong robot learning [3]. We use its four commonly evaluated ten-task suites to characterize a fixed manipulation system, rather than measuring continual learning or forgetting. The choice of ten trials per task follows the sample count used in SmolVLA’s simulation study [2]. Matching that count alone does not reproduce its training or evaluation conditions. We specify the remaining conditions in Section IV.

III. SYSTEM

A. Observation and Prior Information

At step t , the policy receives

$$o_t = (\{I_t^c, D_t^c, K^c, T_{WC,t}^c\}_{c \in \{a,w\}}, r_t), \quad (1)$$

where a and w denote the fixed and wrist cameras, I is RGB, D is metric depth, K is calibration, and T_{WC} maps camera coordinates into the world frame. Robot measurements r_t include end-effector pose, joint state, gripper state, and available force/torque measurements. The grasp-place adapter uses pose, joint positions, and gripper width; contact execution can access the broader robot measurement interface.

These are simulated, calibrated measurements. In particular, camera extrinsics and proprioception are supplied by the simulator adapter, and depth is not an estimate from monocular RGB. The policy observation types contain no object pose labels, segmentation IDs, BDDL predicates, reward, or task-success flag. Task identifiers and initial-state indices remain with the evaluator; reset passes the instruction and a nonsemantic episode token. This is a software interface property supported by code inspection, not a claim of adversarial isolation.

Offline information is represented by a prior collection $\mathcal{P}$. It includes object texture histograms, dimensions derived from public collision geometry, robot geometry, class-specific handling rules, and motion thresholds. The evaluated controller is therefore $a_t = \pi(o_{\leq t}, \ell; \mathcal{P})$, where ℓ is the instruction. No action-policy fitting or demonstration loading occurs in this evaluation. The detector’s pretraining and the controller’s manual development remain part of the system’s knowledge.

B. Constrained Language Compilation

The compiler maps supported language to

$$\mathcal{G}(\ell) = (s_1, \dots, s_M), \quad s_j = (k_j, e_j^{\text{src}}, e_j^{\text{dst}}, \rho_j). \quad (2)$$

Here k_j is a skill, the entity bindings specify source and destination, and ρ_j represents a placement relation or instance selector. The vocabulary contains pick, place-on, place-in, relative placement, stack, open, close, push, turn-on, and

turn-off. For example, a two-object basket instruction becomes two successive pick/place pairs. A drawer instruction can produce an articulation segment followed by a manipulation segment.

Bindings retain relations such as *next to*, *between*, or an ordering among visible instances. They do not encode the simulator’s object identity. The complete program undergoes capability checks before motion, including valid skill arguments and consistency of held-object state. Compilation success only establishes representational support; it does not establish physical executability. The deployed grammar is tailored to known LIBERO phrases and an allow-listed entity vocabulary. An optional semantic-resolver interface exists, but no language-model resolver is configured in this run.

C. Visual Grounding and Geometric State

Each calibrated pixel $\tilde{u} = (u, v, 1)^\top$ with valid depth contributes

$$x_W = R_{WC}(D(u, v)K^{-1}\tilde{u}) + p_{WC}. \quad (3)$$

Workspace filtering, support estimation, and connected components yield candidate point sets. The pipeline fits observed bounds and uses texture/size priors to classify and complete partial observations. The gallery computes a color similarity

$$b(h, h_k) = \sum_i \sqrt{h_i h_{k,i}} \quad (4)$$

between normalized HSV histograms. Its score combines color and shape terms, with configured weights 0.4 and 0.6, a softmax temperature of 0.13, and additional class-dependent footprint gates. Grounding DINO supplies language-conditioned boxes that are converted to 3-D hypotheses using depth. Thus a detector box is a proposal, rather than an oracle object mask.

Cross-camera hypotheses with matching labels are clustered in 3-D. For a cluster, the implemented fusion weights are proportional to $\max(q_i, 0.05)\sqrt{n_i}$, where q_i is confidence and n_i is point count. The default center-distance threshold is 4.5 cm. These thresholds are empirical operating choices; nearby repeated objects can still produce ambiguous associations.

The state retains both observed support/bounds and completed geometry. This distinction matters when an object is partially visible: a size prior can stabilize a center estimate, but should not be mistaken for measured free space. Geometric selectors resolve the intended instance from the available candidates and reference objects. At selected phase transitions, new views are requested and checked against prior observations to reduce hand-occlusion-induced jumps. Specialized fixture and cavity routines refine support surfaces, opening regions, and admissible approach geometry.

D. Grasping, Transport, and Placement

Consecutive manipulation skills share a grasp–place controller. Its phases include acquisition, pregrasp motion, approach, finger closure, retention checking, lift, transport, descent, release, and retreat. Transitions combine pose tolerances, measured gripper width, visual evidence, and bounded

retry logic. Different grasp modes account for exterior pinch, rim pinch, and interior expansion; mode choice depends on the object and observed accessibility. These are mechanical templates with specialized branches.

For target pose (p^*, R^*) , the basic servo emits

$$a_t = \text{clip}_{[-1,1]} \begin{bmatrix} (p^* - p_t)/s_p \\ \text{Log}(R^* R_t^\top)^\vee / s_R \\ g_t \end{bmatrix}. \quad (5)$$

The general grasp–place defaults are $s_p = 0.05$ m and $s_R = 0.25$ rad; cavity handling can select another rotation scale. The nominal pose tolerances are 8 mm and 0.12 rad. These are normalization scales and transition tolerances, not measured tracking-error bounds or a continuous-time stability guarantee.

The gripper and carried object generally have different centers. Let $\hat{d}_t$ be the estimated world-frame vector from the end effector to the carried-object reference point. For desired object location p_o^* , placement planning uses the geometric relation

$$p_{ce}^* = p_o^* - \hat{d}^*, \quad \hat{d}^* = \Delta R \hat{d}_t \quad (6)$$

when the planned wrist motion rotates the held geometry by ΔR . The implementation adds support, release-height, aperture, and fixture-specific corrections to this relation. Offset estimates may come from precontact geometry or refreshed RGB-D evidence; their origin is retained in the execution trace. This allows inspection of placement decisions without assuming that all offset estimates are equally reliable.

Release is followed by sensor-based verification. A local geometric check can reject a placement even when the benchmark predicate has already been true, or accept a relation that the benchmark does not recognize. Section V-C quantifies this distinction rather than using controller completion as the reported success metric.

E. Contact Skills and Sequential Execution

The dispatcher separates consecutive grasp–place steps from atomic contact skills. Drawer and door interactions use observed fixture geometry to construct approach, engagement, motion, release, and retreat stages. Knob and push skills use their own target detectors and motion templates. Shared episode state includes an initial robot-pose reference and, where applicable, an observed drawer anchor for subsequent placement.

Contact evidence combines position error, lack of measured progress, and changes in end-effector force. One implemented guard requires a minimum contact duration and bounded position error, together with either a stalled error reduction or sufficient force change. Such a guard indicates a candidate contact condition; it is not the task predicate. Specialized post-contact visual checks assess the resulting fixture configuration.

Local recovery is bounded by phase counters and geometric checks. Examples include releasing a rejected grasp, reacquiring a visible handle, or performing a short clearance motion after stalled drawer interaction. These routines preserve the distinction between a robot-side mechanical signal

TABLE I
DECLARED CONDITIONS FOR THE EVALUATED SYSTEM.

Property	Setting
Policy observations	Fixed and wrist RGB-D, camera calibration, robot measurements
Offline priors	Public asset textures/dimensions, robot geometry, hand-written skill rules
Action generation	Geometric state machines and clipped Cartesian increments
Learning during run	None; frozen Grounding DINO checkpoint
Scene truth in policy	No object pose labels, segmentation IDs, or BDDL predicates in observation types
Success feedback	External latch; no score-driven policy stopping
Coverage	Known benchmark assets and implemented instruction grammar

and an estimate of task completion. Their hand-selected parameters are part of the fixed controller configuration evaluated below.

Several fixture skills refine the grounded object box into physical contact geometry. A complete visible burner disk anchors the stove support centre; public plate thickness supplies the offset from the visible disk to the load-bearing surface. That unoccluded estimate is retained across sequential placements. For an open microwave, visible vertical control-panel and side-wall surfaces, together with the roof height, constrain the cavity and hinge frame through public physical dimensions. A flat tabletop crop or a roof without supporting vertical structure is insufficient for this fit.

Placement also accounts for the hand’s withdrawal. The microwave approach aligns the payload with the cavity width and uses a lateral entry path. Repeated basket placements receive separate diagonal landing centres. Before withdrawing from a basket, the controller opens the jaws to a measured clearance; a bounded open-wrist tilt clears tall bodies near the palm. All such proposals use language, RGB-D estimates, and robot measurements; the external task predicate is unavailable to their selection or execution.

IV. EXPERIMENTAL PROTOCOL

A. Tasks, Initial States, and Runtime

The evaluation contains $4 \times 10 \times 10 = 400$ episodes: Spatial, Object, Goal, and Long, ten tasks each, and official initial-state indices 0–9. Long is named `libero_10` in the implementation. Global random generators and the environment configuration use seed 7. The environment adapter preserves or replays the task reset stream when starting at a later initial-state index, because resetting can affect fixture configuration beyond the saved joint state.

We use two 256×256 RGB-D cameras and a Panda with operational-space pose control. A ten-step settling period precedes scored actions. The Spatial/Object/Goal/Long action budgets are 220/280/300/520, matching the numerical budgets in the OpenVLA evaluation script [11]. Our environment seed and success-handling logic differ from that script; its environment utility uses seed 0. The run uses Ubuntu 22.04, Python 3.12, PyTorch 2.11.0 with CUDA

12.8, MuJoCo 3.3.2 [12], robosuite 1.4.0 [13], and hf-libero 0.1.4 on one RTX 5090.

All 400 episodes are newly executed against one immutable source snapshot. Task processes execute independently under a dispatcher with up to eight workers. Each worker starts from the same declared seed and evaluates the ten official initializations of one task in index order. The source manifest, per-task configuration, episode coverage, and scoring mode are checked after the campaign. Development records are kept separately from this complete run.

B. External Scoring and Internal Completion

Let q_t be the official success predicate observed by the evaluator. With policy-controlled stopping time τ capped at budget H , the recorded outcome is

$$Y = \bigvee_{t=1}^{\tau} q_t \vee q_{\text{final}}, \quad \tau \leq H. \quad (7)$$

The final query also handles a zero-action stop. Reward and success do not enter the policy observation. The evaluator continues after a success signal until the controller requests stopping or the action budget expires. The score therefore represents *ever reaching* the predicate during this rollout, not necessarily satisfying it at the final frame.

Each record additionally stores terminal controller state, step count, elapsed time, failure message, and geometric traces. The reported success rate is $\hat{p} = N^{-1} \sum_i Y_i$. Equal task sample counts make the overall episode average equal to the macro-average over tasks in the completed run. We retain all planned failures; no outcome-based episode retry or selection is used. Controller-internal retries remain part of a single episode.

C. Scope of the Evidence

The completed campaign passes its record-coverage validation: 400 unique episodes, fixed configuration, existing recorded videos, and matching deployed source checksums. Tables describing our measurements are generated from a frozen projection of the original records, with hashes retaining the connection to their source lines. Published comparison values are transcribed from the cited tables and stored separately from our episode records. Our rates summarize the specified initial states; per-task counts retain the sample size and contribution of each task to the aggregate.

V. RESULTS AND ANALYSIS

A. Comparison with Published Policy Results

Table III compares our measurements with published LIBERO success rates, retaining the original papers’ reported averages. Our implementation is the only system evaluated in the present campaign.

The table specifies the distinct sensing and policy-training settings. The OpenVLA study uses filtered demonstrations [1]. SmolVLA trains across all 40 tasks and evaluates ten trials per task [2]. The selected OpenVLA-OFT setting uses filtered demonstrations and evaluates 500 trials per suite [10]. ANCHOR uses calibrated RGB-D, asset priors,

TABLE II

OBSERVED SUCCESS AND EXECUTION COST. SR USES THE EXTERNAL EVER-SUCCESS PREDICATE. [†]ELAPSED-TIME MEDIAN USES NEWLY EXECUTED EPISODES ONLY; STEPS INCLUDE REUSED RECORDS. TIMES INCLUDE THE RECORDED EPISODE EXECUTION OVERHEAD UNDER CONCURRENT EVALUATION.

Suite	Success	SR (%)	Mean steps	Median s [†]
Spatial	93/100	93.0	147.1	68.1
Object	98/100	98.0	195.3	59.5
Goal	91/100	91.0	180.6	71.5
Long	78/100	78.0	317.0	97.2
Overall	360/400	90.0	210.0	69.5

and the scoring rule in Eq. (7). These differences make the table a comparison of reported system outcomes under their stated conditions, rather than a controlled ranking.

Long has the lowest success rate for every listed method, including ANCHOR, motivating a closer examination of our multi-stage failures.

B. Aggregate and Task-Level Performance

The system succeeds in 360 of 400 episodes, for an overall success rate of 90.0% (Table II). Suite success rates are 93.0% for Spatial, 98.0% for Object, 91.0% for Goal, and 78.0% for Long.

Figure 2 shows the task-level distribution. 24 tasks have ten successful recorded trials. Long 09, placing a mug inside a microwave and closing it, succeeds in 0/10 trials. Long 08, placing both moka pots on the stove, succeeds in 7/10 trials. These tasks combine object placement with confined access, shared supports, or fixture interaction; their individual outcomes provide information that is lost in the overall average.

Drawer-related tasks also remain difficult: Goal task 03, opening the top drawer and placing a bowl inside, succeeds in 3/10 trials; Spatial task 04, retrieving a bowl from a top drawer, succeeds in 7/10; and Long task 03, placing a bowl in a bottom drawer and closing it, succeeds in 7/10. These complete task records identify difficult scenarios and stopping reasons; they do not establish articulation as the cause of failure.

C. Controller Completion Is a Different Outcome

Table IV crosses terminal controller state with external success. In 7 episodes the controller reports completion while the evaluator reports failure. Conversely, 37 episodes receive external success without controller completion. The combined disagreement is 44/400 (11.0%). The latter group includes cases where a predicate is reached before a later local failure or budget exhaustion. Because the recorder retains an ever-success latch, these records do not imply successful final-state verification.

Using local completion as the benchmark label would change the measured outcome; feeding external success to the controller would change its information interface. We record both values separately. The records lack first-success

time and a separate final-only label, so they cannot quantify changes in final-state success from continued execution or the runtime saved by an early-stop evaluator.

D. Recorded Failure Categories

Table V groups the 40 externally unsuccessful episodes using their persisted status and failure message. The grouping gives controller completion/external failure and global-budget exhaustion precedence; remaining messages are classified by phase timeout, grasp/retention rejection, sensor-verification rejection, or another failure. This is a reproducible description of the recorded stopping reason, not a manually verified taxonomy of physical root causes.

A phase timeout may arise from an inaccessible target, calibration error, occlusion, or contact resistance. Likewise, a rejected sensor verification may reflect an actual placement error or perception uncertainty. Determining these causes requires frame-level or geometric-trace analysis. The current evidence supports identifying where execution terminates, while leaving causal attribution open.

E. Execution Cost and Qualitative Context

The median newly executed episode takes 69.5 seconds. Mean action counts range across suites as shown in Table II. Long episodes average 317.0 actions, compared with 195.3 for Object. These costs include controller execution through its own stopping condition; they are not time-to-first-success. Recorded elapsed times mix perception, simulation, rendering, and video overhead while workers compete for the same CPU and GPU. They should not be interpreted as real-robot control frequency.

The run reports 0 runtime exceptions. Dispatcher wall time is 77.1 minutes for the campaign containing 400 new and 0 reused episodes. Figure 3 provides visual context from actual recordings. The upper row shows a successful Object task-00 episode. The lower row is the earliest externally unsuccessful Goal task-03 episode in the official index order, which exhausts its action budget. Selection is stated to make the figure interpretable as illustration, rather than a representative sampling estimate or evidence of a particular failure mechanism.

VI. DISCUSSION AND LIMITATIONS

Explicit geometry and specialized skills solve a substantial fraction of the selected episodes under declared asset and calibration assumptions. Texture and size priors encode known objects; grammar rules encode known language forms; and specialized controllers reflect manual development. These resources matter when interpreting the published-policy comparison.

a) Generalization and novelty. The run covers known assets, language forms, and one simulated embodiment with calibrated depth. It does not evaluate transfer to unseen assets, language, perturbed calibration, or physical hardware, and provides no held-out history of controller development. The rates characterize the deployed benchmark system. Its geometric operations and state machines are established

TABLE III

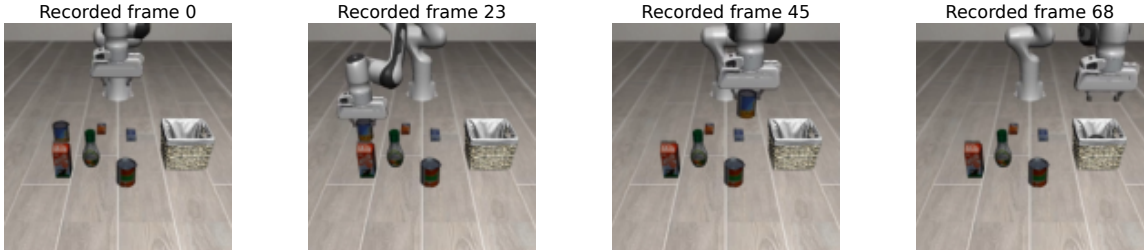
LIBERO SUCCESS RATES (%). PUBLISHED ROWS CONTAIN THE REPORTED MEANS FROM OPENVLA TABLE 12 [1], SMOLVLA TABLE 2 [2], AND OPENVLA-OFT TABLE I [10]; THEY ARE NOT RERUNS BY US. THE OPENVLA-OFT ROW USES THE FULL SETTING WITH ADDITIONAL INPUTS AND FILTERED DEMONSTRATIONS. THE FINAL ROW IS MEASURED IN THIS WORK. ALL METHODS RECEIVE LANGUAGE INSTRUCTIONS; SENSING, PRIORS, TRAINING, AND EVALUATION PROTOCOLS DIFFER AS DESCRIBED IN THE TEXT.

Method	Sensing and action-policy setup	Spatial	Object	Goal	Long	Average
<i>Published results (reported means)</i>						
Diffusion Policy [1]	Single RGB; policy trained from scratch per suite	78.3	92.5	68.3	50.5	72.4
Octo [1]	Single RGB; pretrained policy fine-tuned per suite	78.9	85.7	84.6	51.1	75.1
OpenVLA [1]	Single RGB; pretrained policy fine-tuned per suite	84.7	88.4	79.2	53.7	76.5
SmolVLA (0.45B) [2]	RGB/state; VLM-initialized policy trained across suites	90.0	96.0	92.0	71.0	87.3
OpenVLA-OFT [10]	Two RGB views + state; policy fine-tuned per suite	97.6	98.4	97.9	94.5	97.1
<i>Measured in this work</i>						
ANCHOR	Two RGB-D views + state; asset priors, fixed controller	93.0	98.0	91.0	78.0	90.0

Spatial	10/10	8/10	10/10	9/10	7/10	9/10	10/10	10/10	10/10	10/10
Object	10/10	9/10	10/10	10/10	9/10	10/10	10/10	10/10	10/10	10/10
Goal	10/10	10/10	10/10	3/10	10/10	10/10	8/10	10/10	10/10	10/10
Long	10/10	9/10	9/10	7/10	8/10	9/10	9/10	10/10	7/10	0/10
	00	01	02	03	04	05	06	07	08	09
Official task index (within each suite)										

Fig. 2. External successes divided by recorded trials for each official task index. Columns identify different tasks in each suite; darker cells indicate higher success. Counts, rather than rounded percentages alone, show the sample size supporting each result.

Object 00 / init 00: external success



Goal 03 / init 02: action-budget exhaustion

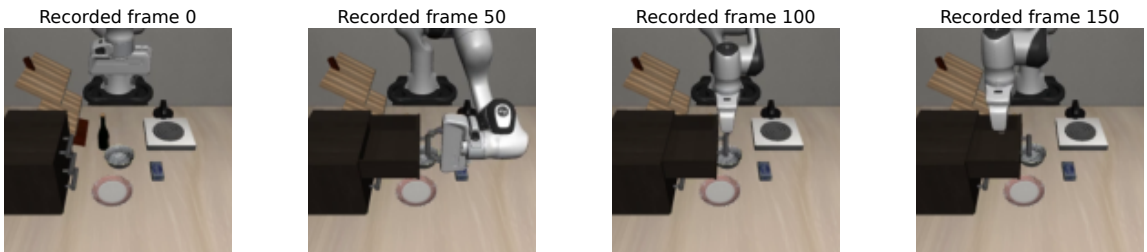


Fig. 3. Actual fixed-camera frames. Top: *pick up the alphabet soup and place it in the basket* (Object 00, initial state 00). Bottom: *open the top drawer and put the bowl inside* (Goal 03, initial state 02). Four uniformly spaced recorder indices are shown (two-action recording stride). Frames are vertically flipped for upright display; left/right is preserved.

TABLE IV

TERMINAL CONTROLLER STATE AND INDEPENDENTLY RECORDED EXTERNAL EVER-SUCCESS. A CONTROLLER FAILURE CAN FOLLOW AN EARLIER BENCHMARK SUCCESS.

Controller terminal state	External success	External failure
Succeeded	323	7
Failed	25	29
Timeout	12	4

TABLE V

OUTCOME GROUPS AMONG EXTERNAL FAILURES. CATEGORIES ARE DERIVED FROM RECORDED CONTROLLER MESSAGES WITH A FIXED PRECEDENCE RULE.

Recorded outcome group	Episodes
Phase timeout	17
Controller completion / external failure	7
Other controller failure	7
Grasp / retention rejection	4
Global action budget	4
Sensor verification rejection	1

techniques; the contribution is their documented integration and empirical characterization.

b) Reliability and interpretation.: Bounded actions and local guards limit the implemented commands, but do not establish collision safety, contact-force safety, or real-world reliability. The narrow ten-trial task samples can also conceal rare failure modes. The observed controller/evaluator disagreement points to a specific measurement improvement: record first-success time and final predicate separately, then evaluate alternative stopping conventions without silently changing the policy’s observations.

c) Reproducibility.: Source and resource checksums, fixed initial-state identifiers, and record-derived figures support reconstruction of this evaluation. The software-boundary counters are implementation-generated records, rather than independent security attestations. The current artifact should therefore be used to reproduce and inspect a concrete system, with the experimental scope stated above.

VII. CONCLUSION

We presented ANCHOR, an asset-informed geometric manipulation baseline that compiles supported language into closed-loop grasp-place and contact skills. Under its declared sensing and prior assumptions, the evaluated system achieves 90.0% external success across 400 recorded episodes. Task-level outcomes and completion disagreements reveal limitations that an aggregate success rate alone would hide. The implementation and record-based analysis make the system’s information requirements, contact behavior, and policy-evaluation interface available for inspection and reproduction.

ACKNOWLEDGMENTS AND AI DISCLOSURE

OpenAI GPT-6-Astra (via Codex) assisted in drafting and editing the text in all sections, preparing the LaTeX source, and writing scripts for the numerical tables and figures.

Results for ANCHOR are computed from existing simulator evaluation records; comparison values are taken from the cited publications. The rollout images are extracted from our recordings; no generative image model was used.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024, version 3, Appendix E, Table 12.
- [2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025, version 1, Table 2.
- [3] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [4] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *arXiv preprint arXiv:2010.01083*, 2020.
- [5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [6] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.
- [7] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” *arXiv preprint arXiv:2307.05973*, 2023.
- [8] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” *arXiv preprint arXiv:2409.01652*, 2024.
- [9] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision*, 2024.
- [10] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025, version 2, Table I.
- [11] OpenVLA Contributors, “LIBERO evaluation script and environment utilities,” <https://github.com/openvla/openvla>, accessed September 2026. Paths: `experiments/robot/libero/run_libero_eval.py` and `libero_utils.py`.
- [12] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [13] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” *arXiv preprint arXiv:2009.12293v3*, 2025.